

WEEK4 Indexes

Indexes — Accelerating Queries with Data Structures

Study notes combining the Stanford lecture video on indexes (Widom) and the CSC404 *Lecture 8: Indexes* slides. Focus: the **user/application perspective** — what indexes are, how they speed up queries, their trade-offs, and how to pick them. Implementation details (how B-trees and hash tables actually work internally) are deliberately out of scope.

1. What Is an Index?

An **index** is a **persistent data structure** built on top of a table's data to **speed up lookups**. It is stored *separately* from the table it indexes but lives alongside the database.

"*Indexes*" and "*indices*" mean the same thing — both are correct. These notes use *indexes*.

Why indexes matter

Indexes are the **primary way to get improved performance out of a database**. As the slides put it, "*the three most important things about a database: performance, performance, performance.*"

The complexity difference

This is the whole point of indexing:

Approach	Time to find a row	Behaviour on a 100-million-row table
Full table scan (no index)	$O(n)$	Reads every row — slow
B-Tree / B+-Tree index	$O(\log n)$	A handful of lookups — fast
Hash index	$O(1)$ (equality only)	Roughly constant — very fast

For huge tables, that's **orders of magnitude** in performance difference.

Two key facts

- Indexes are built on **one or more columns** of a table.
 - The **query optimizer** decides *for each query* whether to actually use a given index — you don't call it directly.
-

2. How an Index Works — The Lookup Picture

Imagine a simple table **T** with three columns. We'll focus on column **A** (animal names, a string) and **B** (an integer); column **C** is anything else.

Sample table **T**

#	A	B	C
1	cat	2	...
2	dog	5	...
3	cow	1	...
4	elk	9	...
5	cat	2	...
6	cat	10	...
7	cow	5	...

The leftmost **#** column is the **tuple number** (a row identifier the database uses internally — like a pointer to the row).

2.1 Index on a single column — point lookups

Build an **index on T.A**. The index lets the query processor ask: "*which tuples have value X in column A?*" and get back the matching tuple numbers **without scanning the whole table**.

Query asked of the index on T.A	Index returns
T.A = 'cow'	tuples 3, 7
T.A = 'cat'	tuples 1, 5, 6

Similarly, an **index on T.B** answers questions about column **B**:

Query asked of the index on T.B	Index returns
T.B = 2	tuples 1, 5

2.2 Range queries

With a tree-based index, you can ask **range** questions too:

Range query on T.B	Index returns
T.B < 6	tuples 1, 2, 3, 5, 7
T.B > 4 AND T.B ≤ 8	tuples 2, 7

2.3 Composite (multi-column) index

You can build an index on **two columns together**, e.g. on (A, B). This handles conditions that constrain both:

Composite query on (T.A, T.B)	Index returns
T.A = 'cat' AND T.B > 5	tuple 6
T.A < 'D' AND T.B = 1	tuple 3

Key idea: With the right index, the database goes **directly** to the qualifying tuples rather than scanning the entire table. On a billion-row table, that's the difference between a microsecond and minutes.

3. Underlying Data Structures

Implementation is out of scope, but you must know **which structure supports which kinds of queries**.

Structure	Equality (A = v)	Range (A < v, BETWEEN, etc.)	Ordering (ORDER BY A)	Lookup cost
B-Tree / B+-Tree (balanced tree)				O(log n)
Hash table				O(1) (constant)

Why the difference?

- A **B-tree** stores keys in **sorted order** along its leaves, so it can walk a range or read items in order naturally.
- A **hash table** scatters keys to buckets via a hash function — there is **no order**, so it cannot answer "less than" or "between" questions, and it can't help with ORDER BY.

Which to pick?

- **B-trees are more flexible** — they handle everything hash indexes do, plus range and ordering. This is the **default** in most database systems.
- **Hash indexes** are slightly faster for **pure equality** lookups, and may be preferred when only equality conditions are used.
- Logarithmic vs. constant is rarely a practical concern even at scale — both are very fast.

Different DBMSs offer different index *types* (e.g., PostgreSQL has **BTREE**, **HASH**, **GiST**, **GIN**, etc.). For ordinary application work, the default B-tree is almost always right.

4. What Queries Benefit From an Index?

An index on column(s) A can help with:

Query pattern	Example	Benefits
Equality lookup	WHERE A = 5	Both tree & hash
Range query	WHERE A BETWEEN 10 AND 50	Tree only
Ordering	ORDER BY A	Tree only (index already stores sorted order, so no extra sort needed)
Joins	JOIN ... ON T.A = S.A	Tree or hash on A in either table

5. Indexes in SQL — Worked Examples

These use the same college-admissions database from earlier lectures: Student(sID, sName, GPA, sizeHS), College(cName, state, enrollment), Apply(sID, cName, major, decision).

5.1 Lookup by primary key

```
SELECT *  
FROM Student  
WHERE sID = 123;
```

With an **index on sID**, the engine jumps straight to that one row. Without it, every student row would have to be scanned.

Many DBMSs auto-create indexes on PRIMARY KEY columns (and often on UNIQUE columns too). So in practice this query is *already* fast — but it's worth checking.

5.2 Multi-condition query

```
SELECT *  
FROM Student  
WHERE sName = 'Mary' AND GPA > 3.9;
```

Three reasonable indexing choices:

Index	Strategy used by the engine
Index on sName	Find all Marys, then filter to those with GPA > 3.9.
Index on GPA (<i>must be a tree index — range condition</i>)	Find all students with GPA > 3.9, then filter to those named Mary.
Composite index on (sName, GPA)	Find both conditions in one index lookup.

Because GPA > 3.9 is an **inequality**, any index covering GPA **must be tree-based** — a hash index won't work for it. sName = 'Mary' is equality, so either type works for sName.

5.3 Join queries

```
SELECT sName, cName  
FROM Student, Apply  
WHERE Student.sID = Apply.sID;
```

Three index strategies, all faster than a brute-force nested loop:

Available index	How it's used
Index on Apply.sID	Scan Student; for each student, jump to matching Apply rows via the index.
Index on Student.sID	Scan Apply; for each application, jump to the matching Student row via the index.
Both indexes	Walk both relations in sorted order of sID and do a merge-style match (often the fastest).

Query planning / query optimization is the area of database systems that picks among these strategies. It is one of the most important — and most interesting — parts of any DBMS, and it's what lets us write *declarative* SQL queries that still run efficiently.

6. Auto-Created Indexes

Most database systems automatically build an index for:

- Every **PRIMARY KEY** declaration (one per table).
- Optionally, every **UNIQUE** constraint (a table may have many of these).

Recall from the constraints lecture: a table has **at most one primary key** but **any number** of additional **UNIQUE** keys.

So if your application declares `sID INT PRIMARY KEY`, you probably *already* have an index on `sID` without writing `CREATE INDEX`.

7. Downsides of Indexes

Indexes are not free. The lecture ranks the three costs from **least to most severe**:

7.1 Extra storage space (*minor*)

The index is a persistent structure stored next to the table. It costs disk space — potentially doubling database size in extreme cases. With modern storage prices, this is rarely a real concern.

7.2 Index-creation overhead (*medium*)

Building an index over an existing large table can take a long time (think: sorting/hashing millions of rows). One-time cost, but it can be painful on big datasets — and it must happen whenever you load data or add a new index.

7.3 Index maintenance on writes (*most significant*)

This is the killer. The index is a **separate** structure, so any time the underlying table changes:

Write op	What must happen
INSERT	Add the new row's keys to every index on the table.
UPDATE (of an indexed column)	Remove the old key from each affected index, add the new one.
DELETE	Remove the row's keys from every index.

Every extra index makes every write slower.

The danger zone: A table that is **modified frequently but queried rarely** can actually be *slowed down* by indexing — maintenance overhead outweighs lookup benefits. **Index decisions are always a cost–benefit trade-off.**

Rule-of-thumb summary

Workload type	Indexing strategy
Read-heavy / analytical	Lots of indexes — queries win, writes are rare
Write-heavy / transactional	Fewer, focused indexes — keep writes fast
Mixed	Profile the workload and index carefully

8. Choosing Which Indexes to Build

The benefit of an index on column A depends on:

1. **Table size** — full scans are expensive only on big tables.

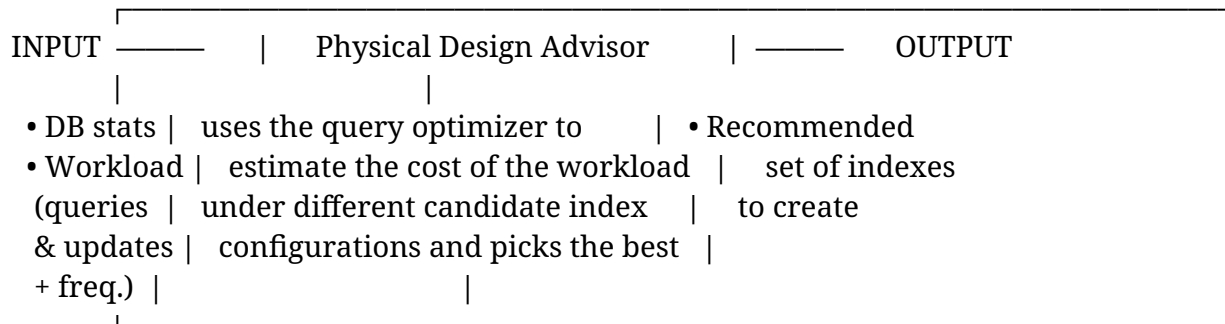
2. **Data distribution / selectivity** — an index helps most when the query returns a **small fraction** of the table. (Indexing a column with only 2 distinct values like Y/N is usually pointless.)
3. **Query frequency** — how often you read using condition on A.
4. **Update frequency** — how often A changes; updates cost more with indexes.

Good indexing candidates

- Columns appearing frequently in WHERE clauses.
- Columns used in JOIN predicates.
- Columns used in ORDER BY.
- **High-selectivity** columns (many distinct values).

Physical Design Advisor

Most major DBMS vendors ship a tool — a **physical design advisor** — to recommend indexes automatically.



How it works internally:

1. Take the **workload** (typical queries + updates + how often each runs) and **database statistics** (table sizes, distributions).
2. Try various **candidate index configurations**.
3. For each configuration, ask the **query optimizer** to estimate the cost of running the workload (without actually executing it).
4. Pick the configuration whose estimated cost is lowest — i.e. where index benefits most outweigh maintenance costs.

Without a design advisor, **you do this analysis manually**: profile your queries and updates, identify hot WHERE/JOIN/ORDER BY columns, and weigh read benefit against write cost.

Reading what the optimizer actually does

Use **EXPLAIN** (or EXPLAIN ANALYZE) to see whether your indexes are actually being used:

```
EXPLAIN SELECT * FROM Student WHERE sName = 'Mary';
```

If the plan says *Index Scan*, the index is working. If it says *Seq Scan* (sequential scan), the optimizer chose to skip the index — usually because the query isn't selective enough, or the index isn't useful for the predicate.

9. SQL Syntax for Indexes

The SQL standard syntax is straightforward.

Create a single-column index

```
CREATE INDEX idx_student_name  
ON Student(sName);
```

Create a composite (multi-column) index

```
CREATE INDEX idx_student_name_gpa  
ON Student(sName, GPA);
```

Column order matters in composite indexes. An index on (sName, GPA) is great for queries that filter on sName first (or both), but is **not useful** for queries that filter only on GPA. Put the most-selective / most-frequently-queried column first.

Create a unique index (also enforces a uniqueness constraint)

```
CREATE UNIQUE INDEX idx_student_email  
ON Student(email);
```

This does two things at once:

1. Builds an index on email.
2. **Enforces** that no two rows have the same email — inserting a duplicate raises an error.

Drop an index

```
DROP INDEX idx_student_name;
```

Each DBMS adds its own variations (e.g., CREATE INDEX ... USING BTREE in MySQL/PostgreSQL, or CREATE INDEX ... INCLUDE(...) for covering indexes in SQL Server). Check your DBMS's docs for specifics.

10. Summary

- An **index** is a **persistent data structure** stored beside a table that lets the database **locate matching rows quickly** instead of scanning the whole table.
- Indexes are the **primary way to improve database performance** — often by **orders of magnitude**.
- The two foundational structures are **B-trees** (equality + range + ordering, $O(\log n)$) and **hash tables** (equality only, $O(1)$).
- An index helps with WHERE filters, JOINS, and ORDER BY. Composite indexes help with multi-column conditions; column order matters.
- Most systems auto-create indexes for **PRIMARY KEY** and (often) **UNIQUE** columns.
- Indexes have **costs**: storage (minor), creation overhead (medium), and — most importantly — **maintenance on every INSERT/UPDATE/DELETE**.
- The right index set depends on **table size**, **data distribution**, **query frequency**, and **update frequency**. Use a **physical design advisor** or EXPLAIN to guide decisions.
- Build indexes on columns that appear in WHERE/JOIN/ORDER BY, especially **high-selectivity** ones.

Quick reference

Concept	Key fact
What is an index	Persistent data structure for fast lookups
Tree index	B-Tree / B+-Tree; equality, range, ordering; $O(\log n)$
Hash index	Equality only; $O(1)$
Auto-indexed	PRIMARY KEY, often UNIQUE
Helps with	WHERE =, WHERE </BETWEEN, JOIN ON, ORDER BY
Hurts	Writes (INSERT/UPDATE/DELETE

Concept	Key fact
	maintenance)
Picking indexes	Frequent + selective columns; profile workload
Tooling	Physical design advisor, EXPLAIN / EXPLAIN ANALYZE

SQL cheat sheet

```
CREATE INDEX idx_name    ON Table(col);
CREATE INDEX idx_name    ON Table(col1, col2); -- composite
CREATE UNIQUE INDEX idx_name ON Table(col);    -- + uniqueness
DROP INDEX idx_name;
EXPLAIN <your query>;                -- see the plan
```

*Indexes are deceptively simple from the outside but enormously consequential for real database performance. The art is not building **lots** of indexes — it's building the **right** indexes for your workload.*